Project - High Level Design

On

MULTI-MODEL INSURANCE CREATOR

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	S. JEZREEL	EN22IT301085
2	KRITAGYA TIWARI	EN22IT301048
3	SADHANA YADAV	EN24CA5030146
4	CHANDRESH BHATI	EN24CA50040
5	SHIVKUMAR SOLANKI	EN24CA5030157

Project Number:DO-40

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year:2025-26

Group Name: 02D12

Table of Contents

S.No.	Title	Page No.
1	Introduction	4
1.1	Scope of the Document	4
1.2	Intended Audience	5
1.3	System Overview	5-6
2	System Design	7
2.1	Application Design	8-9
2.2	Process Flow	9
2.3	Information Flow	9
2.4	Components Design	9
2.5	Key Design Considerations	10
2.6	API Catalogue	11-14
3	Data Design	15
3.1	Data Model	15-17
3.2	Data Access Mechanism	17
3.3	Data Retention Policies	18
3.4	Data Migration	18-19
4	Interfaces	20-21

5	State and Session Management	22-23
6	Caching	24
7	Non-Functional Requirements	25
7.1	Security Aspects	25-26
7.2	Performance Aspects	27-28
8	References	29

PROJECT OVERVIEW

Insurance Hub is a cloud-deployed insurance management platform built with Flask. It allows authenticated users to manage their insurance policies (Health, Term, Vehicle) and leverages Generative AI to explain insurance concepts and generate professional visual infographics — demonstrating the synergy between text-to-text and text-to-image AI models.

1) INTRODUCTION:

Insurance Hub is a full-stack web application designed to manage insurance policies and integrate **Multimodal Generative AI capabilities** for enhanced user experience. The system allows users to securely manage different types of insurance policies such as **Health Insurance, Term Insurance, and Vehicle Insurance**.

In addition to standard policy management features, the application includes an **AI Studio** that utilizes large language models and image generation models to explain insurance concepts and generate visual risk infographics.

The backend of the system is developed using **Python Flask**, while the AI functionality integrates **LLM APIs and text-to-image models**. The platform demonstrates how **Generative AI can be combined with traditional software systems** to provide intelligent and interactive features for users

1.1) SCOPE OF THE DOCUMENT:

This document defines the Low Level Design (LLD) of the Insurance Hub — Multimodal GenAI Insurance Management Application. It describes the internal structure and implementation details of the system, including modules, database schema, workflows, and integration with external AI services. The scope covers both the insurance policy management features (CRUD operations, authentication, and user data handling) and the AI Studio module, which utilizes text-to-text and text-to-image models to generate policy explanations and risk infographics. This document serves as a technical guide for understanding how the system processes user inputs, interacts with APIs, and produces outputs.

1.2) INTENDED AUDIENCE:

This document is intended for developers, testers, project evaluators, and system administrators who are involved in the design, development, deployment, or evaluation of the Insurance Hub — Multimodal GenAI Insurance Management Application. It helps developers understand the system architecture, module interactions, and API integrations, assists testers in validating system functionality and workflows, and enables evaluators to assess the correctness and effectiveness of the technical design. Additionally, it is useful for students and learners to understand the practical implementation of multimodal Generative AI systems, including the integration of LLM-based text generation and image generation models within a Flask-based web application.

1.3) SYSTEM OVERVIEW:

The **Multimodal Insurance Creator** is a Generative AI–powered system designed to transform **textual insurance concepts into both detailed explanations and visual representations**. The primary objective of the system is to demonstrate how **multimodal artificial intelligence models can work together** to create informative and visually engaging content related to insurance policies and risk concepts.

The application combines **text generation models (Large Language Models)** and **image generation models** through external APIs to produce two types of outputs from a single user prompt:

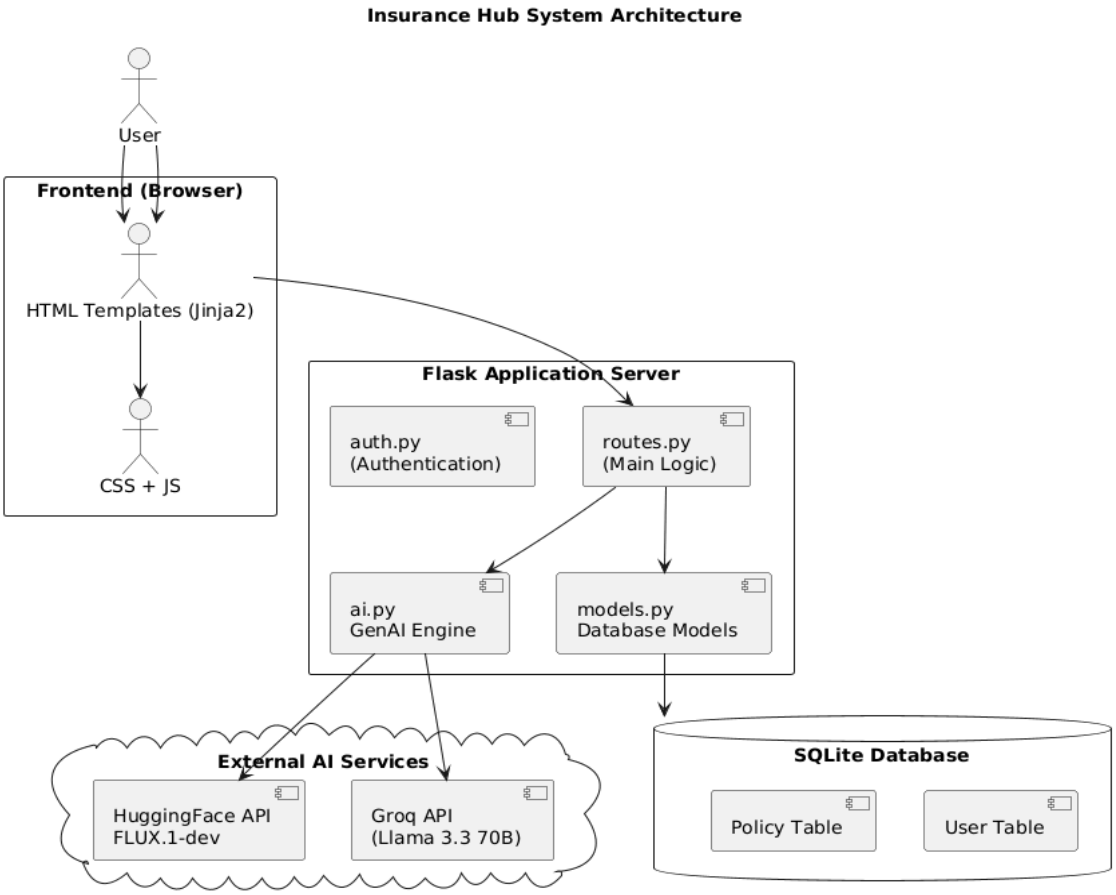
1. **Textual Explanation** — generated using a Large Language Model (LLM) that explains insurance concepts in a structured and easy-to-understand format.

2. **Visual Infographic** — generated using an AI image model that converts the same prompt into a professional infographic or conceptual visualization.

This dual-output approach illustrates the **synergy between text-to-text and text-to-image generative models**, allowing users to understand complex insurance topics through both **descriptive information and visual representation**.

1.4 System Architecture Diagram:

The system architecture of the Insurance Hub application defines the overall structure of the system and explains how different components interact to deliver insurance management and multimodal AI capabilities. The architecture follows a layered web application design, where each layer performs a specific function such as user interaction, application processing, data storage, and AI service integration. This modular design improves maintainability, scalability, and security of the system



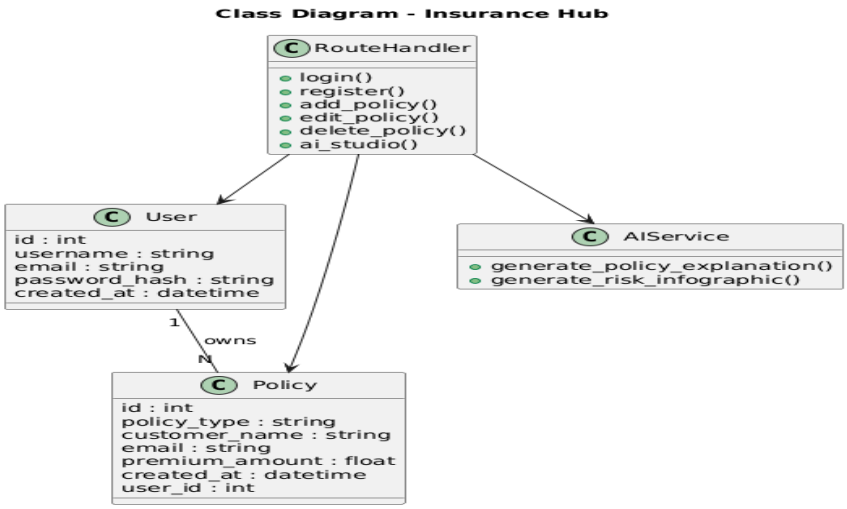
2. SYSTEM DESIGN

2.1 Application Design

The Insurance Hub application follows a **modular and layered architecture** based on the Flask framework. The system is divided into three main layers: **presentation layer, application layer, and data layer**. The presentation layer consists of HTML templates rendered using Jinja2, providing an interactive user interface. The application layer handles business logic through Flask routes, authentication modules, and AI integration logic. The data layer uses SQLite with SQLAlchemy ORM to manage user and policy data. Additionally, the system integrates external AI services, including **LLM-based text generation and image generation APIs**, to provide multimodal outputs. This modular design ensures **scalability, maintainability, and clear separation of concerns.**

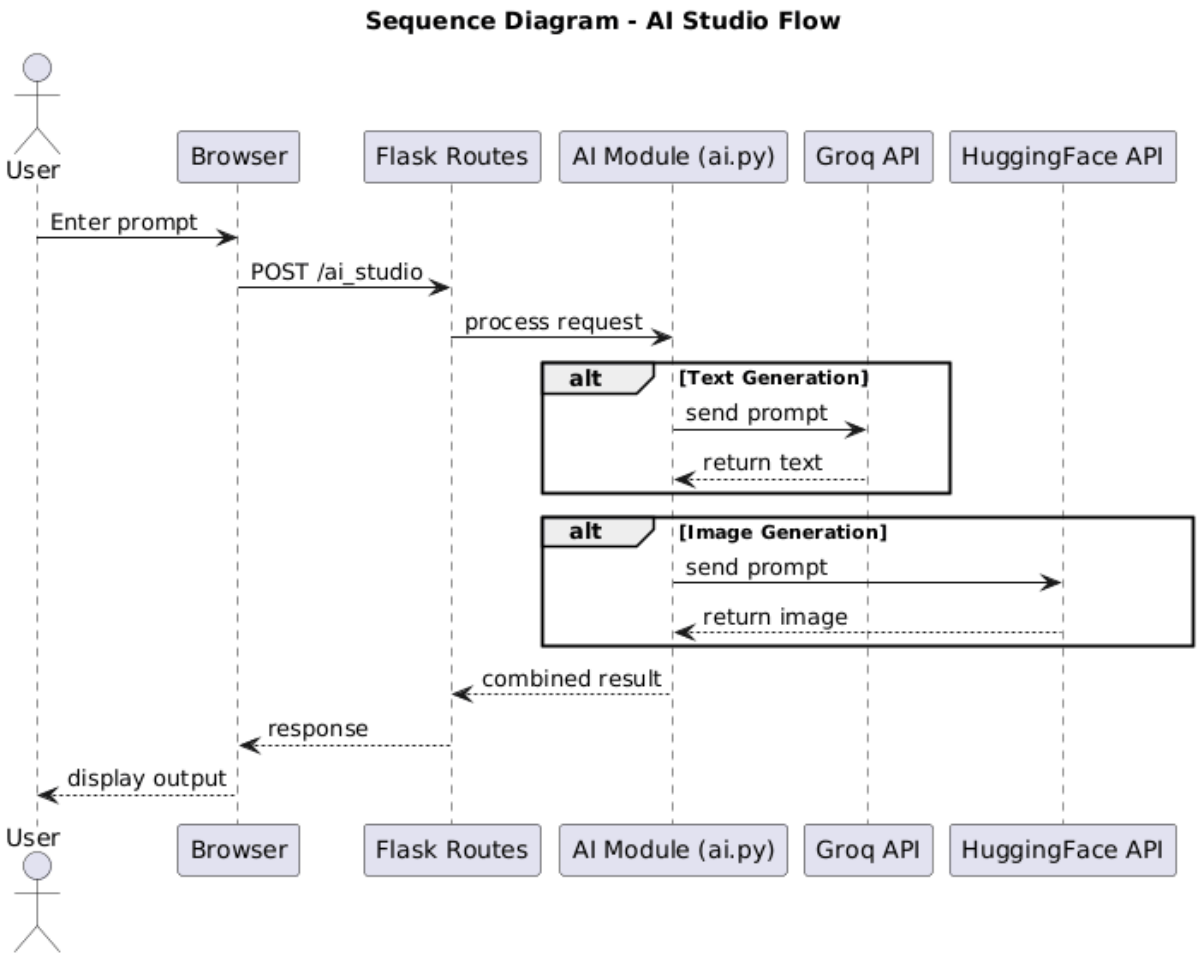
2.1.1 Class Diagram

The class diagram represents the structural design of the Insurance Hub system. It includes core classes such as User and Policy for database representation, along with RouteHandler and AIService for application logic. The relationships show how users own policies and how thesysteminteracts with AI services for generating text and image outputs.



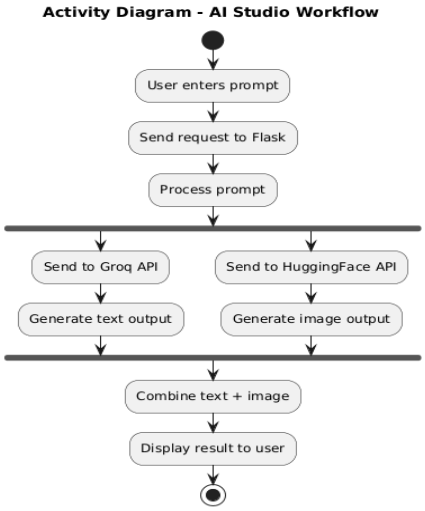
2.1.2 Sequence Diagram

The sequence diagram illustrates the interaction flow when a user requests AI-generated content. It shows how the request moves from the browser to Flask routes, then to the AI module, and further to external APIs like Groq and Hugging Face. The responses are processed and returned to the user interface.



2.1.3 Activity Diagram

The activity diagram represents the workflow of the AI Studio feature. It begins with user input, followed by prompt processing, then splits into parallel activities for text and image generation. After both processes complete, the outputs are combined and displayed. It highlights the multimodal processing nature of the system.



2.2 Process Flow

The system process begins when a user interacts with the web interface. For policy management, the user performs operations such as adding, updating, or deleting policies, which are processed by Flask routes and stored in the database. For AI Studio features, the user provides a prompt related to an insurance concept. The system processes this input and sends it to the AI module. The prompt is then forwarded to two external services: the **LLM API for text generation** and the **image generation API for infographic creation**. The responses from both services are received, processed, and combined into a structured output, which is then displayed to the user. This flow ensures smooth interaction between user input, backend processing, and AI services.

2.3 Information Flow

The information flow in the system involves the movement of data between different components. User input data is first captured through the frontend interface and sent to the backend via HTTP requests. The backend validates and processes the data before either storing it in the database (for policy operations) or forwarding it to external AI APIs (for AI Studio features). The AI-generated responses are returned to

the backend, formatted appropriately, and sent back to the frontend for display. Sensitive data such as user credentials are securely handled using password hashing and session management. This structured flow ensures

data integrity, security, and efficient communication between components.

2.4 Component Level Design

2.4.1 Flask Application Components

The backend consists of **five main modules.**

Module	File	Responsibility
App Factory	__init__.py	Initializes Flask, database, login manager
Authentication	auth.py	Register, login, logout
Main Logic	routes.py	Dashboard, CRUD operations, AI Studio
AI Engine	ai.py	LLM and image generation
Data Models	models.py	Database tables

2.5 KEY DESIGN CONSIDERATIONS

Several important design considerations were taken into account while developing the system. **Security** is ensured through password hashing, session management, and user data isolation. **Scalability** is achieved by designing the system in a modular way and supporting deployment through Docker and Kubernetes. **Performance** is optimized by using efficient API calls and handling AI responses asynchronously where possible. **Usability** is improved through a responsive interface and clear output formatting. **Reliability** is maintained by handling API failures and providing fallback mechanisms. These considerations ensure that the system is robust, secure,

and user-friendly.

2.6 API CATALOGUE

The Insurance Hub application integrates both **external AI APIs** and **internal application APIs** to deliver its core functionalities. The external APIs are responsible for **multimodal AI processing**, while the internal APIs (Flask routes) handle **authentication, policy management, and AI request handling**.

2.6.1 LLM API (TEXT-TO-TEXT GENERATION)

The system uses the Groq API to generate structured insurance explanations through a Large Language Model.

- **Service:** Groq API
- **Model:** Llama 3.3 70B Versatile
- **Purpose:** Generate detailed and structured insurance policy explanations
- **Module Used:** ai.py → generate_policy_explanation()

Input

User prompt (e.g., *"Explain health insurance benefits"*)

Processing

- Prompt is enhanced with a system context (Indian insurance domain)
- Sent to LLM via API request

Output

- Structured text response:
    - Overview

- Key Benefits
- Important Information
- Premium Range

2.6.2 IMAGE GENERATION API (TEXT-TO-IMAGE)

The system uses the Hugging Face Inference API for generating insurance infographics.

- **Service:** Hugging Face InferenceClient
- **Model:** FLUX.1-dev
- **Purpose:** Generate professional insurance risk infographics
- **Module Used:** ai.py → generate_risk_infographic()

Input

- Text prompt describing visual concept
  (e.g., *"Infographic showing health insurance risk coverage"*)

Processing

- Prompt sent to image model
- Model generates image (PIL format)
- Image converted to **Base64 format**

Output

- Embedded image displayed directly in UI (no external URL dependency)

2.6.3 Internal Application APIs (Flask Routes)

The application defines multiple **internal APIs using Flask routes**, which handle user interaction, data processing, and AI integration.

Authentication APIs

Endpoint	Method	Description
/login	POST	Authenticates user and starts session
/register	POST	Registers a new user account
/logout	GET	Logs out the current user

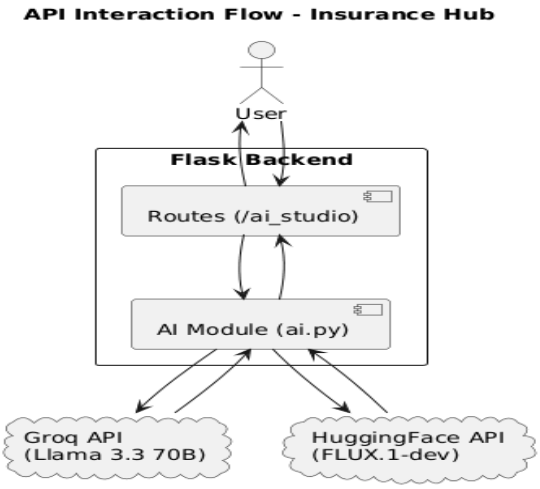
Policy Management APIs

Endpoint	Method	Description
/add_policy	POST	Adds a new insurance policy
/edit_policy	POST	Updates existing policy details
/delete_policy	POST	Deletes a policy
/	GET	Displays user dashboard with policies

AI Studio API

Endpoint	Method	Description
/ai_studio	POST	Handles AI requests (text + image generation)

2.6.4 API INTERACTION FLOW



3) DATA DESIGN

3.1 Data Model

The data model of the *Insurance Hub* application is designed using a **relational database approach** with SQLite as the primary database and SQLAlchemy as the ORM. The system consists of two core entities: **User** and **Policy**. The User entity stores account-related information such as username, email, and password hash, while the Policy entity stores insurance details such as policy type, customer name, premium amount, and creation date. A **one-to-many relationship (1:N)** exists between User and Policy, where one user can manage multiple policies. This model ensures **data consistency, normalization, and efficient storage** while supporting user-specific data isolation.

3.1.1 Database Design

Database: SQLite

ORM: SQLAlchemy

1. User Table

Column	Type	Description
id	Integer	Primary key
username	String(100)	Unique username
email	String(150)	Unique email
password_hash	String(256)	Hashed password
created_at	DateTime	Account creation time

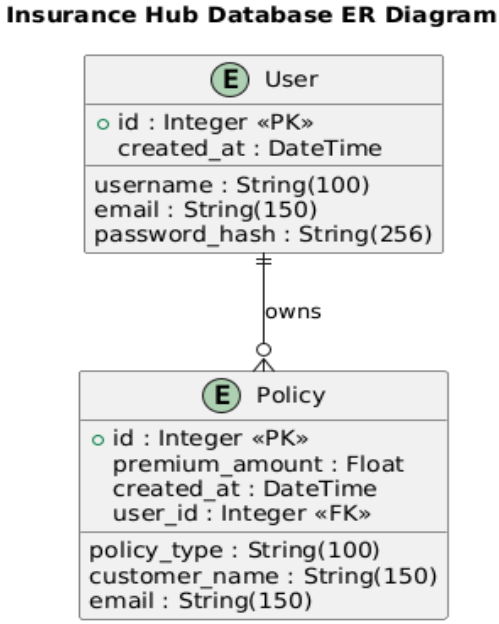
2 Policy Table

Column	Type	Description
id	Integer	Primary key
policy_type	String(100)	Health / Term / Vehicle
customer_name	String(150)	Policy holder name
email	String(150)	Contact email
premium_amount	Float	Annual premium in INR
created_at	DateTime	Policy creation time
user_id	Integer	FK → User.id

3.1.2 ER Diagram

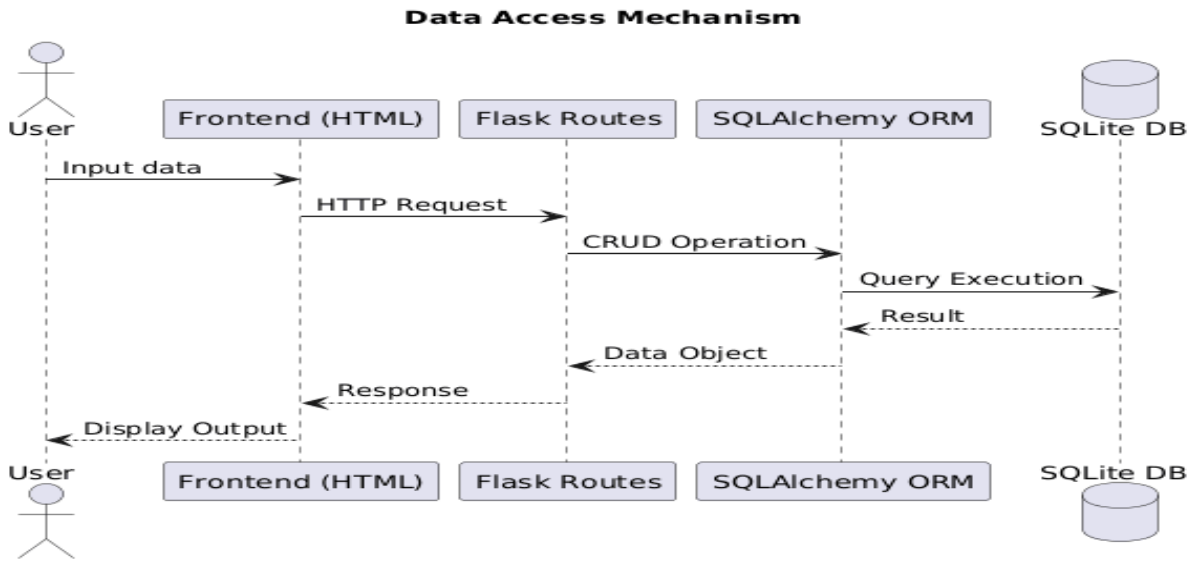
The **Entity Relationship (ER) Diagram** represents the structure of the database used in the **Insurance Hub application**. It shows the entities, their attributes, and the relationships between them. In this system, there are two main entities: **User** and **Policy**.

The **User entity** stores information about registered users such as **id, username, email, password_hash, and created_at**. The **Policy entity** stores insurance policy details like **policy_type, customer_name, premium_amount, and created_at**.There is a **one-to-many (1:N) relationship** between **User and Policy**, meaning one user can have multiple policies, but each policy belongs to only one user.This relationship is implemented using the **user_id foreign key** in the Policy table.



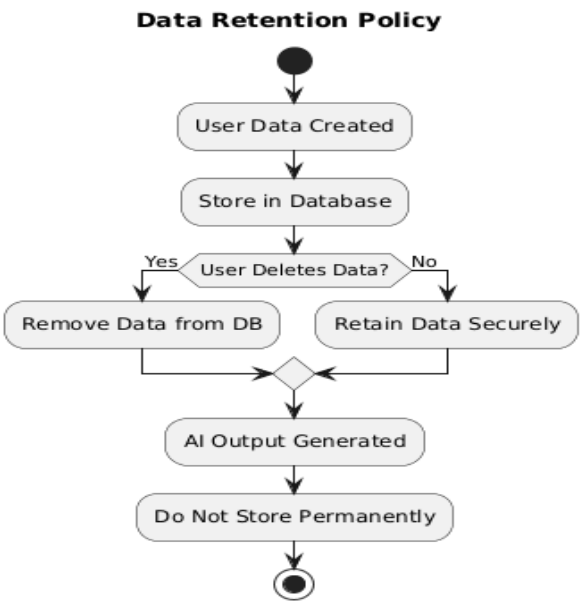
3.2 Data Access Mechanism

The application uses **SQLAlchemy ORM** to manage all database interactions in an object-oriented manner. Instead of writing raw SQL queries, the system performs CRUD operations through Python classes defined in models.py. The Flask routes (routes.py) handle requests and interact with the database via ORM methods. User authentication is managed using Flask-Login, ensuring secure session handling. This mechanism provides **abstraction, security, and maintainability** while simplifying database operations.



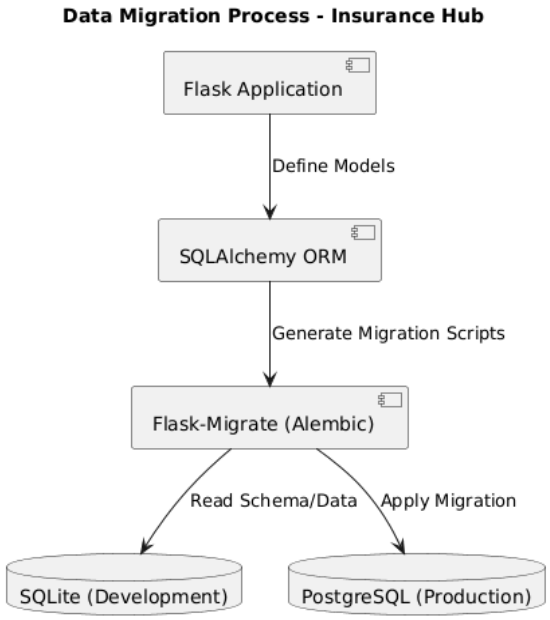
3.3 Data Retention Policies

The system follows a **controlled data retention approach**. User account and policy data are stored persistently in the database until the user updates or deletes them. Sensitive data such as passwords are stored in **hashed format** using secure hashing algorithms. The system ensures **user data isolation**, meaning each user can access only their own policies. AI-generated outputs such as policy explanations and infographics are **not stored permanently**, reducing storage overhead and improving data privacy. This approach ensures **security, minimal data storage, and user control over data**.



3.4 Data Migration

The application supports flexible data migration due to the use of **SQLAlchemy ORM and modular database design**. While SQLite is used during development, the system can be migrated to more scalable databases such as PostgreSQL for production environments. Migration tools like Flask-Migrate can be used to update schema changes without data loss. This ensures that the system remains **scalable, portable, and adaptable** to future requirement


Data Migration Process - Insurance Hub
Flask Application
Define Models
SQLAlchemy ORM
Generate Migration Scripts
Flask-Migrate (Alembic)
Read Schema/Data
Apply Migration
SQLite (Development)
PostgreSQL (Production)

4) INTERFACES

The Insurance Hub application provides multiple interfaces to enable interaction between users, backend services, and external AI systems. It includes user interfaces for interaction, backend interfaces for processing, and external APIs for multimodal AI features. These interfaces ensure seamless communication, efficient data exchange, and smooth integration of text and image generation services.

4.1 User Interface

The user interface is developed using HTML, CSS, and Jinja2 templates, providing a responsive and interactive experience. It includes pages such as login, registration, dashboard, and AI Studio. Users can manage policies and access AI features through structured layouts and navigation. UI elements such as forms, buttons, flash messages, and tab-based views improve usability. Dynamic rendering ensures real-time updates, while the clean design enhances user engagement and simplifies interaction with system functionalities.

4.2 Application Interface

The application interface is implemented using Flask routes that act as endpoints for handling HTTP requests. These routes manage authentication, policy CRUD operations, and AI processing. Each route processes input data, applies business logic, and returns responses to the frontend. The system follows a structured request-response model, ensuring efficient communication between components. This design simplifies integration, improves maintainability, and ensures that each functionality is handled in a modular and organized manner.

4.3 External API Interface

The system integrates external APIs for multimodal AI functionality. The Groq API is used for generating structured text explanations, while the Hugging Face API is used for generating risk infographics. These APIs are accessed securely using tokens stored in environment variables. They enable the system to process user prompts and generate meaningful

outputs, combining text and images to enhance understanding of insurance concepts.

4.4 Data Exchange Format

Data exchange between system components is carried out using HTTP requests and responses, primarily in JSON format. User inputs are submitted through forms and processed by the backend. The responses are rendered dynamically using Jinja2 templates. AI APIs also return structured data, which is formatted and displayed to users. This standardized data exchange ensures consistency, reduces complexity, and enables efficient communication between frontend, backend, and external services.

4.5 Error Handling Interface

The system includes robust error handling mechanisms to manage invalid inputs, authentication issues, and API failures. Flash messages and alerts are used to inform users about errors or successful actions. Input validation ensures only correct data is processed, while exceptions are handled gracefully to prevent system crashes. This improves system reliability and provides a smooth user experience by clearly communicating issues and guiding users toward corrective actions.

5) STATE AND SESSION MANAGEMENT

State and session management in the Insurance Hub application ensures secure and consistent user interactions. The system maintains user authentication using sessions while following a stateless architecture for request handling. This approach enables efficient processing, secure access control, and proper user data isolation, ensuring that each user's session is managed independently and securely.

5.1 Session Lifecycle

The session lifecycle begins when a user logs into the system and ends when the user logs out or the session expires. During this period, the session maintains the user's authentication state, allowing access to protected routes such as policy management and AI Studio. This ensures continuity in user interaction without repeated logins. Proper session lifecycle management improves security and provides a seamless user experience throughout the session duration.

5.2 Session Storage

Session data is stored securely using cookies managed by Flask-Login. Only essential information such as user identity is stored, while sensitive data is protected. The use of secure cookies ensures that session data is transmitted safely between the client and server. This approach minimizes security risks while maintaining efficient session handling, allowing users to access system features without compromising data safety.

5.3 Stateless Request Handling

The application follows a stateless architecture where each request is processed independently without storing request-specific data on the server. This improves scalability and simplifies system design. Session information is used only for maintaining user authentication. Stateless processing ensures that the system can handle multiple requests efficiently, making it suitable for cloud-based deployment and distributed environments.

5.4 Authorization Control

Authorization ensures that only authenticated users can access specific features of the system. The application verifies user identity before allowing access to protected routes such as policy management and AI Studio. Unauthorized users are redirected to the login page. This mechanism ensures secure access control, prevents unauthorized usage, and maintains the integrity of user data within the system.

5.5 Session Security

Session security is maintained through secure cookie handling and proper session termination mechanisms. Sessions are invalidated upon logout to prevent unauthorized reuse. Additional measures such as session timeout and protection against session hijacking can be implemented. These practices ensure that user sessions remain secure, protecting sensitive information and maintaining system integrity.

6) CACHING

Caching in the Insurance Hub application is used to improve system performance and reduce load time. While the current system primarily relies on real-time processing, caching is applied to static resources. Future enhancements may include server-side caching to optimize AI responses and database queries, improving efficiency and scalability.

6.1 Static Content Caching

Static resources such as CSS, JavaScript, and images are cached at the browser level to reduce load time. This minimizes repeated server requests and improves page rendering speed. By storing frequently used resources locally, the system enhances user experience and ensures faster navigation across different pages.

6.2 Server-Side Caching

Currently, server-side caching is not implemented in the system. However, it can be introduced using technologies like Redis to store frequently accessed data or AI responses. This would reduce repeated computations and API calls, improving system performance. Implementing server-side caching can significantly enhance efficiency, especially for high-traffic scenarios.

6.3 API Response Handling

AI responses are generated dynamically for each request to ensure accuracy and relevance. Since outputs depend on user input, caching is not currently applied. However, repeated queries could benefit from caching in the future. Efficient handling of API responses ensures minimal delay and accurate output delivery.

6.4 Cache Invalidation

Cache invalidation ensures that outdated or incorrect data is removed or updated. If caching is implemented, proper strategies must be used to maintain data consistency. This ensures users always receive accurate and up-to-date information, preventing issues caused by stale data.

7) NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the overall quality attributes of the Insurance Hub — Multimodal GenAI Insurance Management Application. These requirements focus on how the system performs rather than what it does. They include important aspects such as usability, scalability, reliability, maintainability, availability, and portability. These factors ensure that the application delivers a smooth and efficient user experience while maintaining system stability.

The system is designed to handle real-world usage by supporting multiple users, integrating external AI services, and operating in a cloud-based environment. By using technologies like containerization and modular architecture, the application ensures flexibility, easy maintenance, and high performance. These requirements play a crucial role in making the system robust, scalable, and suitable for future enhancements, especially for multimodal AI-based functionalities.

7.1 Security Aspects

Security aspects ensure the protection of user data, system integrity, and API interactions within the Insurance Hub application. The system implements authentication, data protection, secure API handling, and input validation. These measures prevent unauthorized access, safeguard sensitive information, and ensure secure integration with external AI services.

7.1.1 Authentication Security

User authentication is implemented using Flask-Login, which securely manages login sessions and access control. Only registered users can access protected features such as policy management and AI Studio. The system verifies user identity before granting access, ensuring that unauthorized users cannot interact with the application's core functionalities.

7.1.2 Password Protection

Passwords are stored using secure hashing mechanisms provided by Werkzeug. This ensures that plain-text passwords are never stored in the database. Even if the database is compromised, the hashed passwords cannot be easily decoded. This approach significantly enhances data security and protects user credentials from unauthorized access.

7.1.3 API Key Security

API keys for Groq and Hugging Face are stored securely in environment variables using a .env file. These keys are not exposed in the source code or frontend. This prevents unauthorized usage of external APIs and ensures secure communication with AI services.

7.1.4 Data Isolation

The system enforces strict user data isolation, ensuring that each user can access only their own policies. This is achieved through user-specific queries and session-based access control. Data isolation prevents unauthorized data access and maintains privacy, which is critical for handling sensitive insurance information.

7.1.5 Input Validation

All user inputs are validated before processing to ensure correctness and prevent malicious data. Validation checks help avoid errors and protect against common vulnerabilities such as injection attacks. This improves system stability and enhances overall security by ensuring only valid data is processed.

7.1.6 Secure Deployment

The application is deployed using Docker and cloud infrastructure, ensuring a controlled and secure environment. Sensitive configurations such as API keys and secret keys are managed securely. This reduces the risk of exposure and enhances the overall security of the system during deployment and operation.

7.2 Performance Aspects

Performance aspects focus on ensuring fast response time, efficient resource utilization, and scalability of the Insurance Hub application. The system is optimized to handle user requests, database operations, and AI API interactions efficiently. Proper design and deployment strategies ensure consistent performance under varying workloads.

7.2.1 Backend Efficiency

The backend is built using Flask, a lightweight framework that ensures fast request handling. SQLAlchemy ORM enables efficient database operations, reducing query complexity. The system processes user requests quickly, ensuring minimal delay in operations such as policy management and AI interactions.

7.2.2 API Optimization

AI API calls are optimized by sending structured prompts and handling responses efficiently. The system ensures minimal latency by processing API responses quickly. Efficient integration with AI services allows smooth generation of both text and image outputs without unnecessary delays.

7.2.3 Frontend Optimization

Frontend performance is improved by optimizing static resources such as CSS and JavaScript files. These resources are cached at the browser level, reducing load time. The use of clean UI design and minimal dependencies ensures faster rendering and a smooth user experience.

7.2.4 Scalability Performance

The use of Docker and Kubernetes enables the system to scale dynamically based on user demand. This ensures that performance remains stable even under high traffic conditions. Load balancing and resource allocation help maintain system responsiveness.

7.2.5 Resource Management

The system efficiently utilizes resources such as CPU, memory, and network bandwidth. Proper handling of API calls and database operations ensures that resources are not overused. This results in stable and efficient system performance.

7.2.6 Future Improvements

Future enhancements may include implementing caching mechanisms, asynchronous processing, and load balancing. These improvements will further reduce response time and enhance system scalability. Advanced optimization techniques can make the system more efficient and capable of handling larger workloads.

References

1. Flask (web framework)

Use: Used to build the backend of the application, handle routing, request processing, and integrate all modules like authentication, database, and AI features.

Link:
https://flask.palletsprojects.com/

2. SQLAlchemy

Use: Used for database design and operations. It helps in creating models (User, Policy) and performing CRUD operations without writing raw SQL queries.

Link:
https://docs.sqlalchemy.org/

3. Flask-Login

Use: Used to manage user authentication, session handling, login/logout functionality, and access control for protected routes.

Link:
https://flask-login.readthedocs.io/

4. Groq API

Use: Used for LLM-based text generation in the AI Studio feature. It generates structured insurance explanations using Llama 3.3 70B model.

Link:
https://console.groq.com/docs

5. Hugging Face API

Use: Used for generating AI-based risk infographics from text prompts using the FLUX.1-dev model.

Link:
https://huggingface.co/docs/api-inference

6. Docker

Use: Used to containerize the application, making deployment consistent across different environments such as local machine and cloud servers.

Link:
https://docs.docker.com/